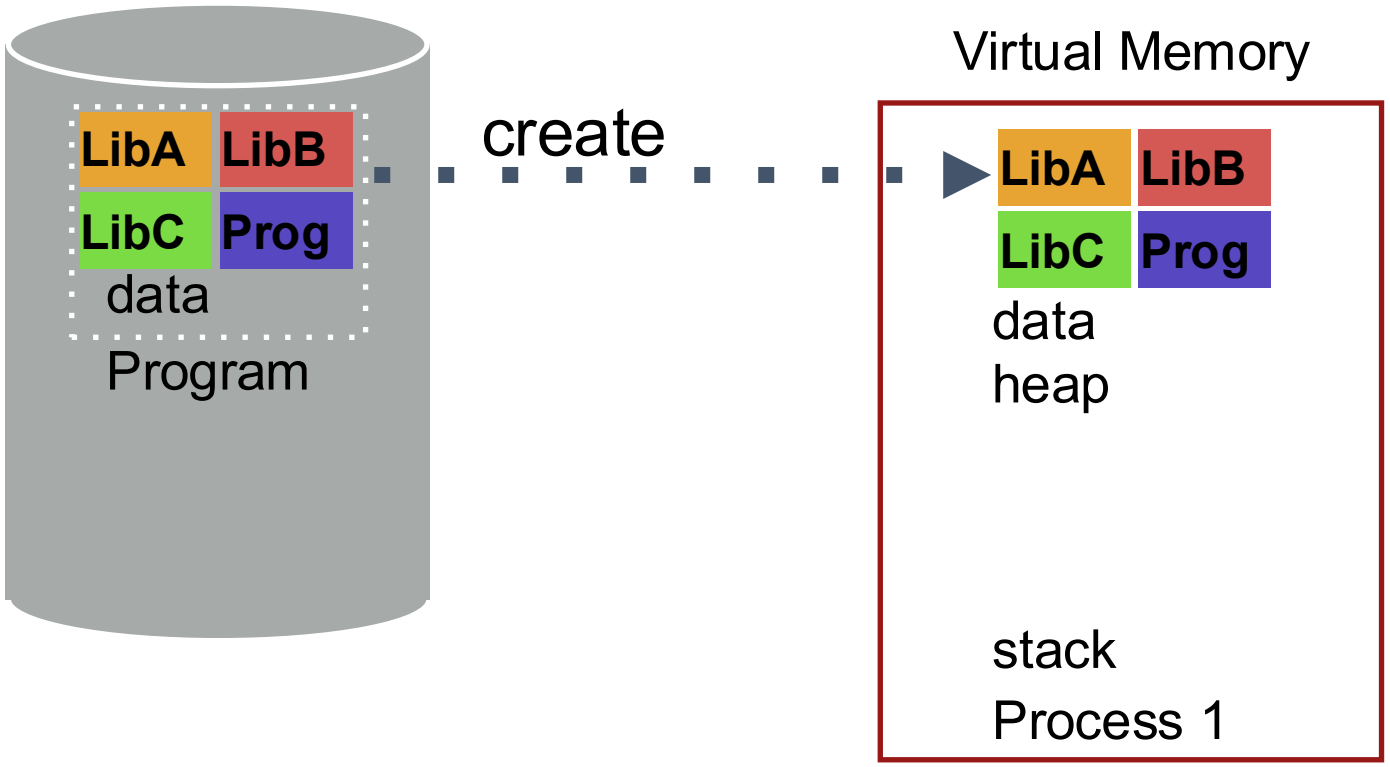
Memory Virtualization

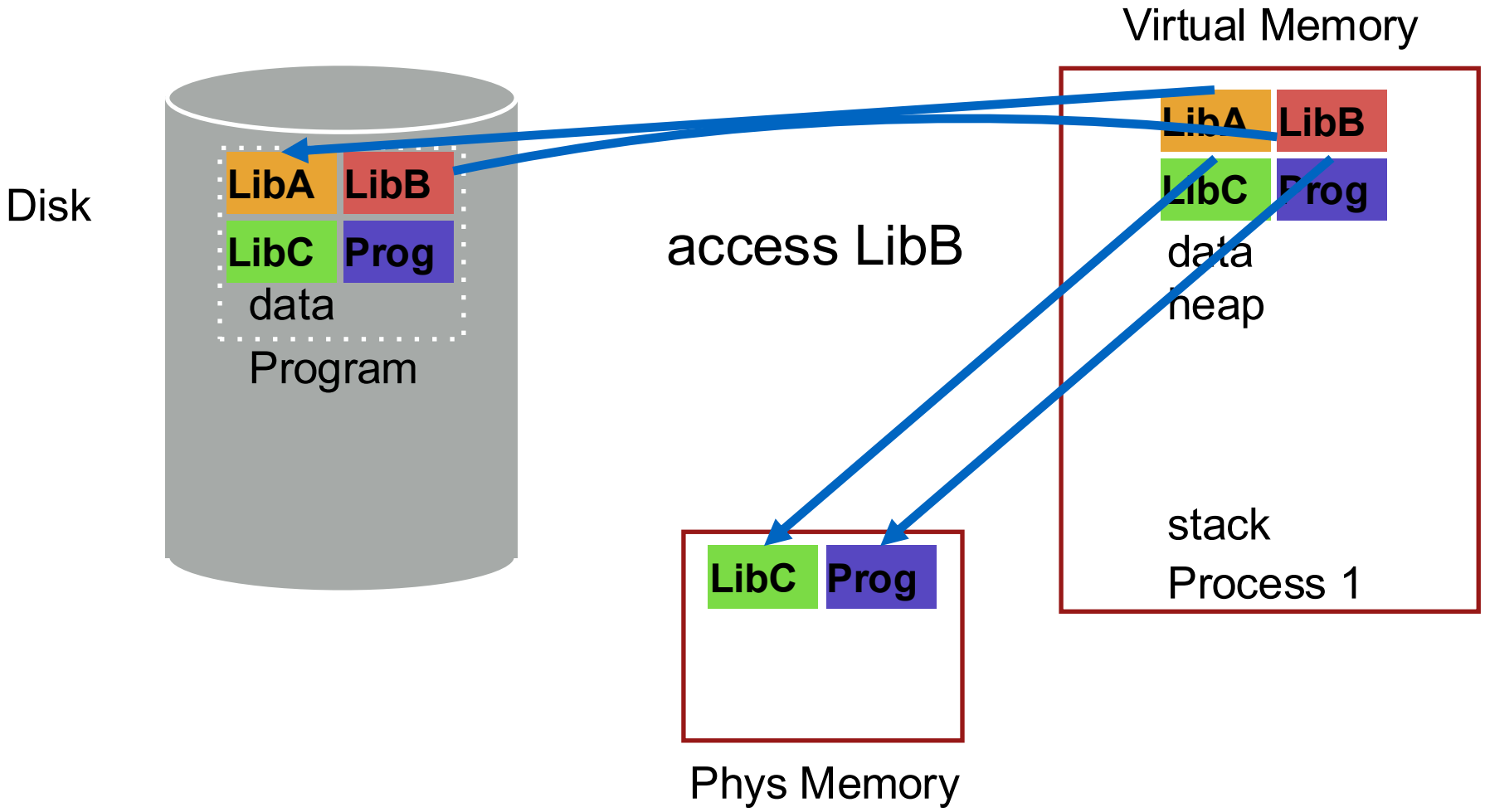


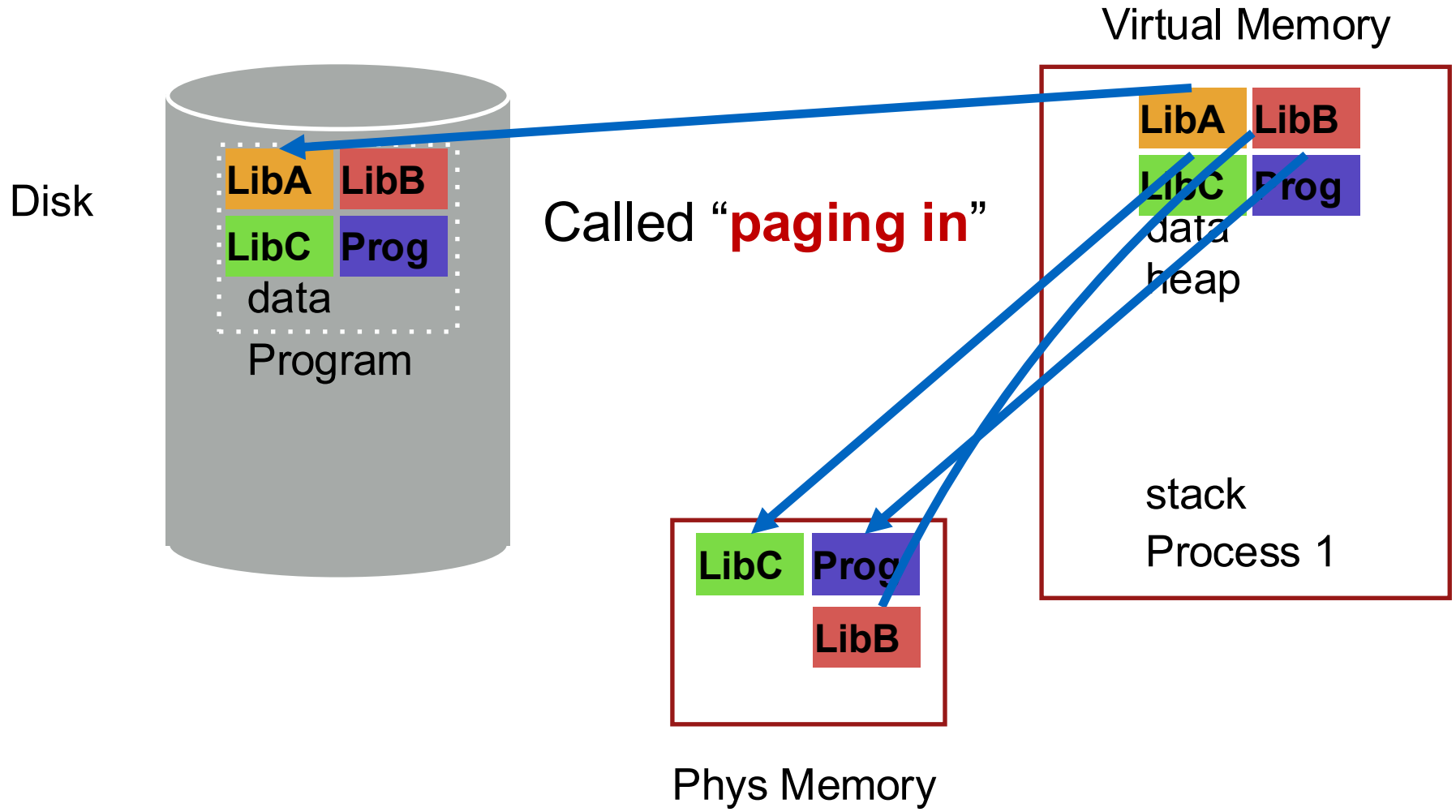
Review



many large libraries, some
of which are rarely/never used

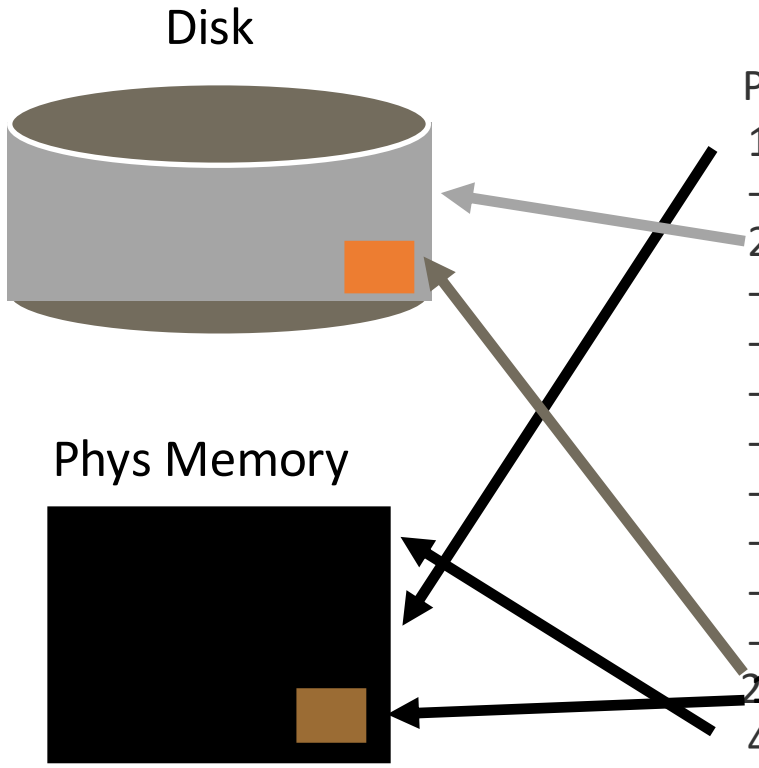
How to avoid wasting physical pages to back rarely used virtual pages?


Virtual Memory
Disk
LibA
LibB
LibC
Prog
data
Program
access LibB
LibA
LibB
LibC
Prog
data
heap
stack
Process 1
LibC
Prog
Phys Memory



(Somewhat confusingly) The idea of maintaining just a limited portion of the virtual address space in physical memory is termed Virtual memory.

The “Present” Bit



PFN	valid	prot	present	
10	1	r-x	1	
-	0	-	-	
23	1	rw-	0	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
-	0	-	-	
28	1	rw-	0	// VPN 0xb
4	1	rw-	1	

What if access vpn 0xb?

Virtual Memory Mechanisms

Hardware and OS cooperate to translate addresses

First, hardware checks TLB for virtual address

- if TLB hit, address translation is done; page in physical memory

If TLB miss...

- Hardware or OS walk page tables
- If PTE designates page is present, then the page is in physical memory
- Else, if PTE valid (but not present), the OS brings the page from the disk into physical memory

Hardware memory access: Control flow

```
1   VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2   (Success, TlbEntry) = TLB_Lookup(VPN)
3   if (Success == True)   // TLB Hit
4       if (CanAccess(TlbEntry.ProtectBits) == True)
5           Offset   = VirtualAddress & OFFSET_MASK
6           PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7           Register = AccessMemory(PhysAddr)
8       else
9           RaiseException(PROTECTION_FAULT)
10  else                    // TLB Miss
11      PTEAddr = PTBR + (VPN * sizeof(PTE))
12      PTE = AccessMemory(PTEAddr)
13      if (PTE.Valid == False)
14          RaiseException(SEGMENTATION_FAULT)
15      else
16          if (CanAccess(PTE.ProtectBits) == False)
17              RaiseException(PROTECTION_FAULT)
```

Hardware memory access: Control flow

```
1   VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2   (Success, TlbEntry) = TLB_Lookup(VPN)
3   if (Success == True)   // TLB Hit
4       if (CanAccess(TlbEntry.ProtectBits) == True)
5           Offset   = VirtualAddress & OFFSET_MASK
6           PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7           Register = AccessMemory(PhysAddr)
8       else
9           RaiseException(PROTECTION_FAULT)
10  else                    // TLB Miss
11      PTEAddr = PTBR + (VPN * sizeof(PTE))
12      PTE = AccessMemory(PTEAddr)
13      if (PTE.Valid == False)
14          RaiseException(SEGMENTATION_FAULT)
15      else
16          if (CanAccess(PTE.ProtectBits) == False)
17              RaiseException(PROTECTION_FAULT)
18          else if (PTE.Present == True)
19              // assuming hardware-managed TLB
20              TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21              RetryInstruction()
```

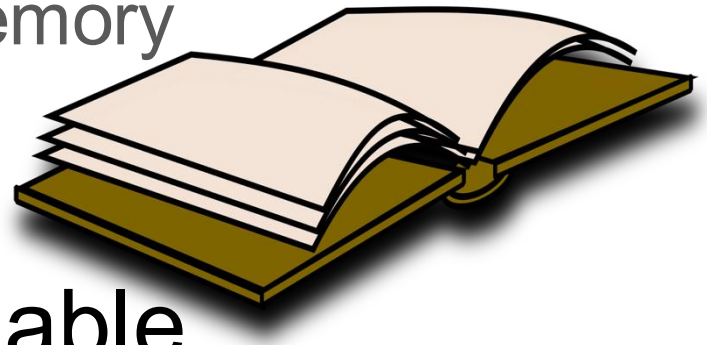
Hardware memory access: Control flow

```
1   VPN = (VirtualAddress & VPN_MASK) >> SHIFT
2   (Success, TlbEntry) = TLB_Lookup(VPN)
3   if (Success == True)   // TLB Hit
4       if (CanAccess(TlbEntry.ProtectBits) == True)
5           Offset   = VirtualAddress & OFFSET_MASK
6           PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
7           Register = AccessMemory(PhysAddr)
8       else
9           RaiseException(PROTECTION_FAULT)
10  else                    // TLB Miss
11      PTEAddr = PTBR + (VPN * sizeof(PTE))
12      PTE = AccessMemory(PTEAddr)
13      if (PTE.Valid == False)
14          RaiseException(SEGMENTATION_FAULT)
15      else
16          if (CanAccess(PTE.ProtectBits) == False)
17              RaiseException(PROTECTION_FAULT)
18          else if (PTE.Present == True)
19              // assuming hardware-managed TLB
20              TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
21              RetryInstruction()
22          else if (PTE.Present == False)
23              RaiseException(PAGE_FAULT)
```

Virtual Memory Mechanisms

If page fault (i.e., `present` bit is cleared)

- Trap into OS (not handled by hardware. Why?)
- If needed, OS selects victim page in memory to replace
  - Write victim page out to disk if modified. Add `modified` (`"dirty"`) bit to PTE
  - Update PTE of the victim page
- OS reads referenced page from disk into memory
- Page table is updated, `present` bit is set
- Process continues execution



Process scheduler should put a runnable process on CPU while page fault serviced

Virtual Memory Policies

Virtual Memory Policies



Goal: Minimize number of page faults

- Page faults require milliseconds to handle (reading from disk)
- Implication: Plenty of time for OS to make good decision

OS has two decisions

- Page selection
  - **When** should a page (or pages) on disk be **brought into** memory?
- Page replacement
  - **Which r**esident page (or pages) in memory should be **moved out** to disk?

Average Memory Access Time (AMAT)

Hit% = portion of accesses that go straight to RAM

Miss% = portion of accesses that go to disk first

Tm = time for memory access

Td = time for disk access

AMAT = (Tm) + (Miss% * Td)

Page Selection (1/3)

When should a page be brought from disk into memory?

Demand paging: Load page only when page fault occurs

- Intuition: Wait until page must absolutely be in memory
- When process starts: No pages are loaded in memory
- Problems: Pay the cost of a page fault for every newly accessed page

Page Selection (2/3)

When should a page be brought from disk into memory?

Pre-paging (anticipatory, prefetching): Load page before referenced

- OS predicts future accesses and brings pages into memory early
- Works well for some access patterns (e.g., sequential)

Page Selection (3/3)

When should a page be brought from disk into memory?

Hints: Combine above with user-supplied hints about page references

- User specifies: may need page in future, don't need this page anymore, or sequential access pattern, ...
- Example: `madvise()` in Unix

Page Selection (3/3)

https://www.man7.org/linux/man-pages/man2/madvise.2.html

```
int madvise(size_t size;
            void addr[size], size_t size, int advice);
```

MADV_NORMAL
No special treatment. This is the default.

MADV_RANDOM
Expect page references in random order. (Hence, read ahead may be less useful than normally.)

MADV_SEQUENTIAL
Expect page references in sequential order. (Hence, pages in the given range can be aggressively read ahead, and may be freed soon after they are accessed.)

MADV_WILLNEED
Expect access in the near future. (Hence, it might be a good idea to read some pages ahead.)

MADV_DONTNEED
Do not expect access in the near future. (For the time being, the application is finished with the given range, so the kernel can free resources associated with it.)

Page Replacement

Which page in main memory should selected as victim?

- Write out victim page to disk if modified ("dirty" PTE bit set)
- If a victim page is not modified (clean), just discard

OPT: Replace page not used for longest time in future

- Advantages: Guaranteed to minimize number of page faults
- Disadvantages: Requires that OS predict the future; Not practical, but good for comparison

OPT Replacement Example

Page reference string: 1,2,3,1,2,4,1,4,2,3, 2

Three pages
of physical memory

OPT

Miss: 1,2,3



Metric:
Miss
count

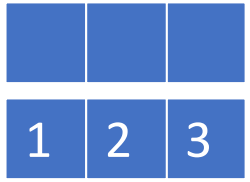
OPT Replacement Example

Page reference string: **1,2,3**,1,2,4,1,4,2,3, 2

Three pages
of physical memory

OPT

Miss: 1,2,3



Metric:
Miss count : 3

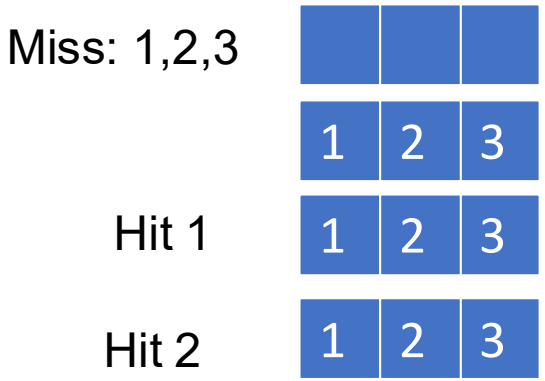
Compulsory
misses

OPT Replacement Example

Page reference string: 1,2,3,**1,2**,4,1,4,2,3, 2

Three pages
of physical memory

OPT

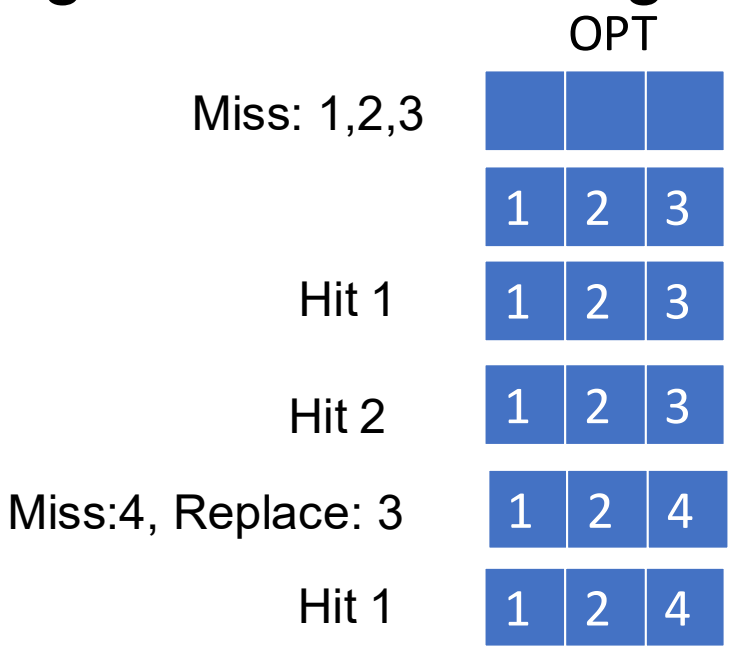


Metric:
Miss count : 3

OPT Replacement Example

Page reference string: 1,2,3,1,2,**4,1**,4,2,3, 2

Three pages
of physical memory

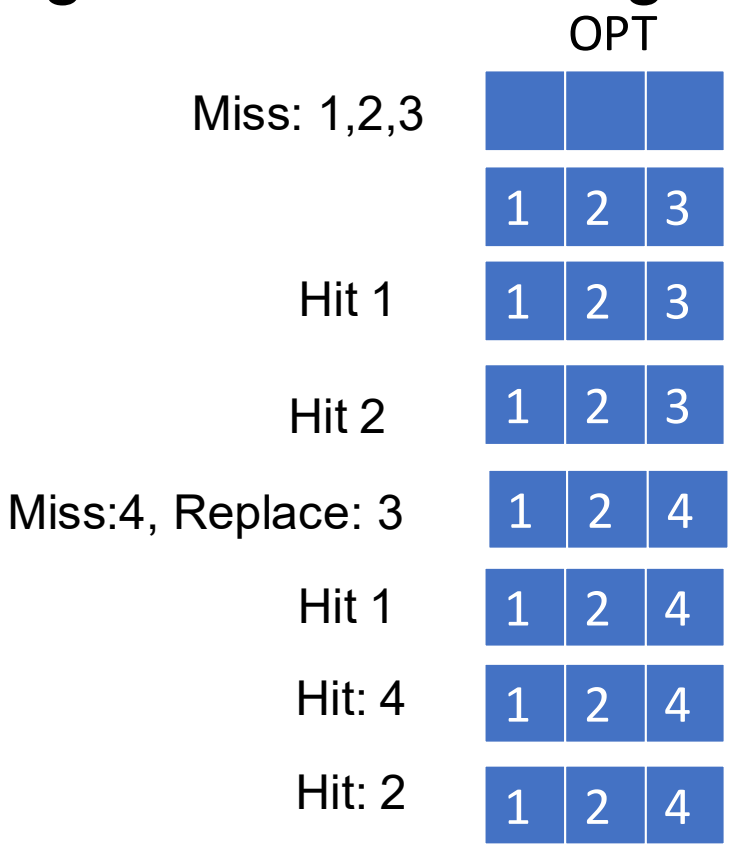


Metric:
Miss count: 4

OPT Replacement Example

Page reference string: 1,2,3,1,2,4,**1**,**4**,**2**,3, 2

Three pages
of physical memory



Metric:
Miss count: 4

capacity

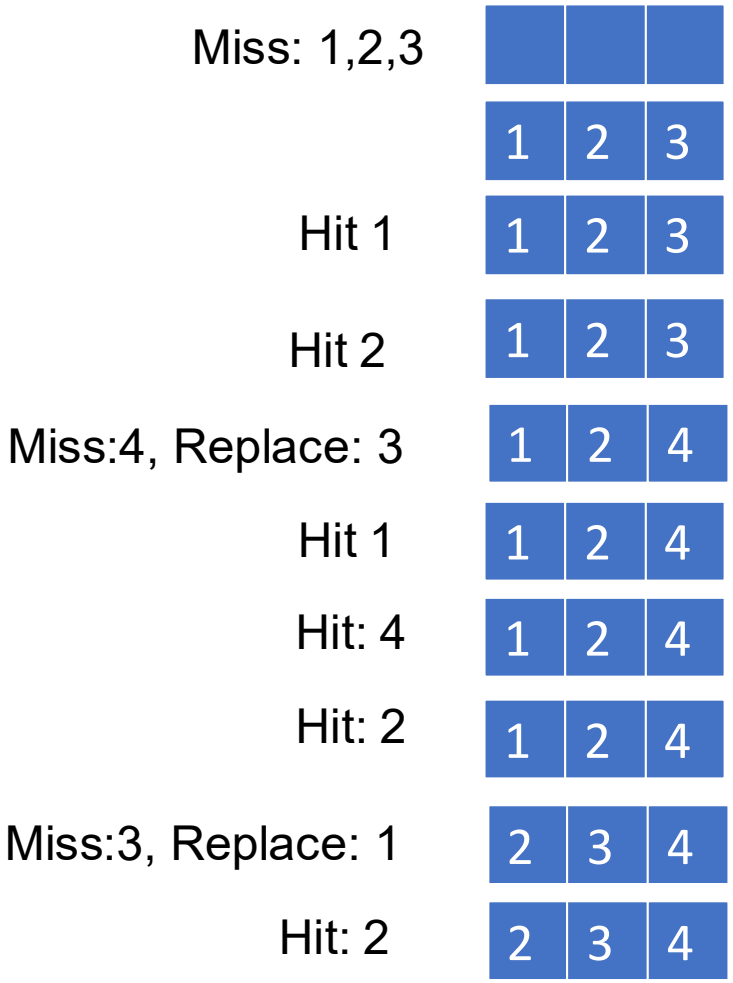
miss

OPT Replacement Example

Page reference string: 1,2,3,1,2,4,**1**,4,2,**3, 2**

Three pages of physical memory

OPT



Metric:
Miss count : 5
AMAT?

5 misses, 4 compulsory misses

AMAT = (Tm) + (Miss% * Td)

Assume Tm = 100ns
Assume Td = 1000000 ns (1millisec)

AMAT = ? 100 + 1000000*(5/11) ns

FIFO

FIFO: Replace page that has been in memory the longest

- Intuition: First referenced long time ago, so maybe we are done with it now?
- Advantages: Fair: All pages receive equal residency; Easy to implement (circular list)
- Disadvantage: Some pages may always be needed

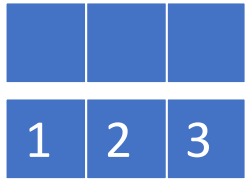
FIFO Example

Page reference string: **1,2,3**,1,2,4,1,4,2,3,2

Three pages
of physical memory

OPT

Miss: 1,2,3



Metric:
Miss count: 3

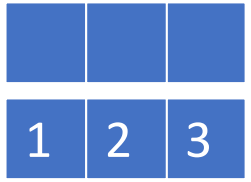
FIFO Example

Page reference string: 1,2,3,**1,2,4**,1,4,2,3,2

Three pages
of physical memory

OPT

Miss: 1,2,3



Hit: 1



Hit: 2



Miss:4, Replace:1



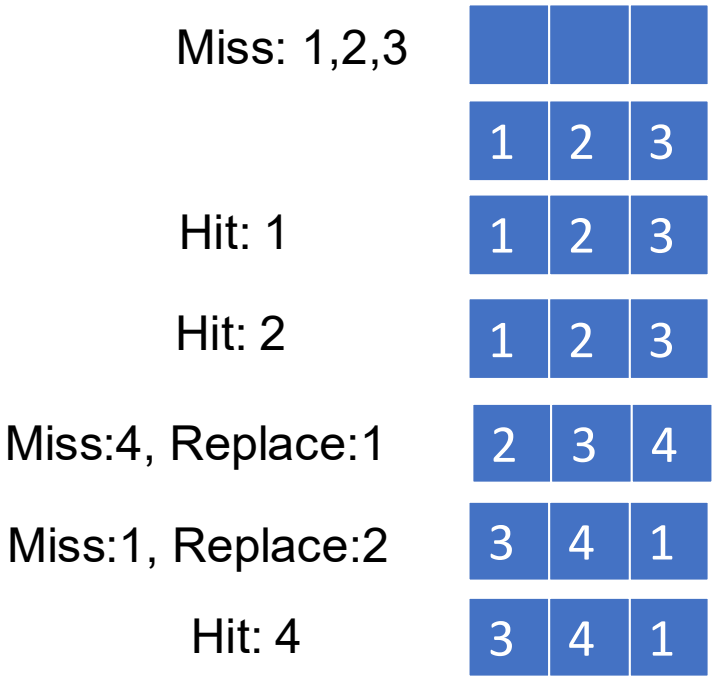
Metric:
Miss count: 4

FIFO Example

Page reference string: 1,2,3,1,2,4,**1,4**,2,3,2

Three pages
of physical memory

OPT



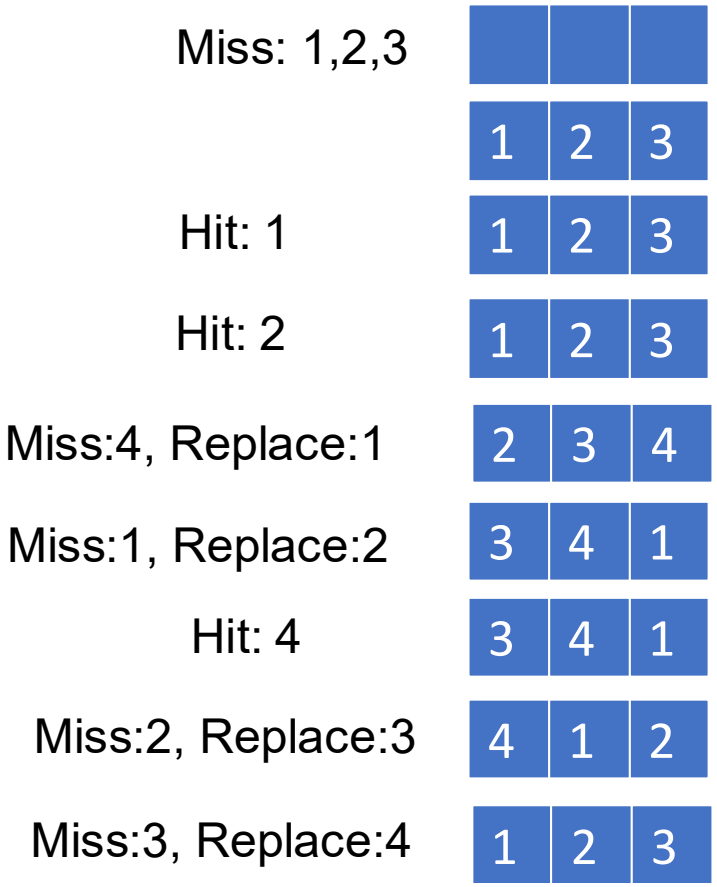
Metric:
Miss count: 5

FIFO Example

Page reference string: 1,2,3,1,2,4,1,4,**2,3**,2

Three pages
of physical memory

OPT



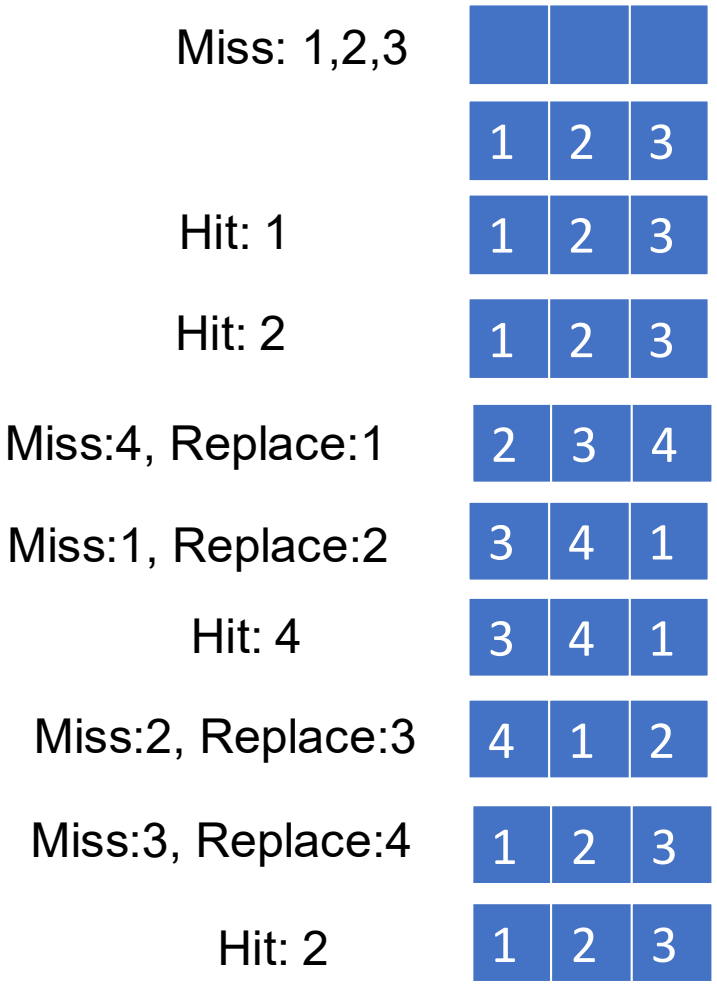
Metric:
Miss count : 7

FIFO Example

Page reference string: 1,2,3,1,2,4,1,4,2,3,**2**

Three pages
of physical memory

OPT



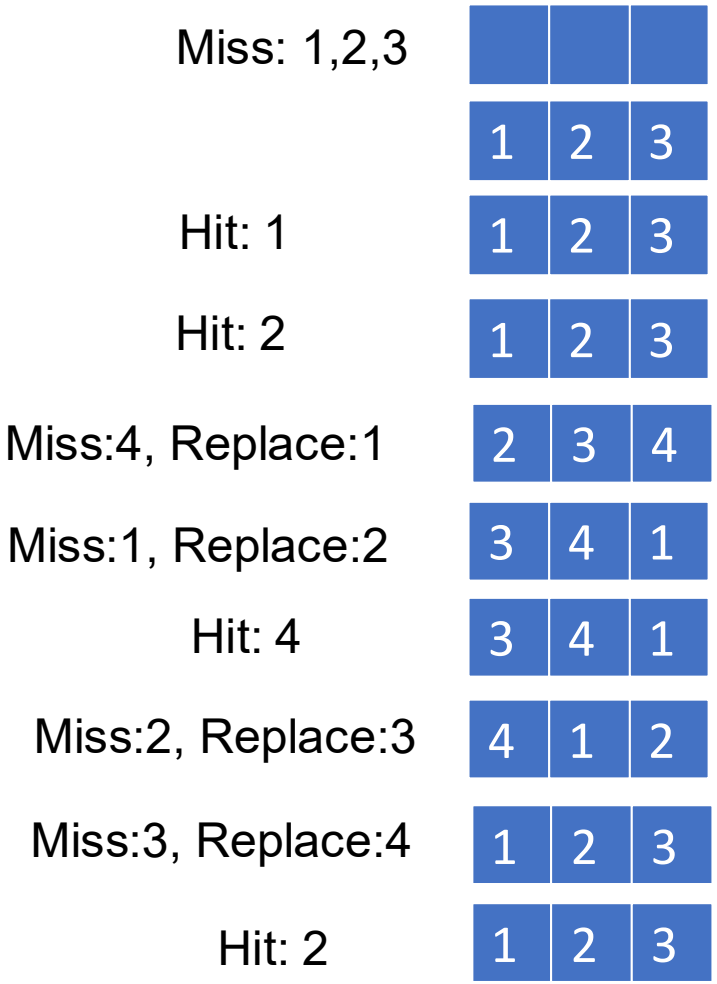
Metric:
Miss count : 7

FIFO Example

Page reference string: 1,2,3,1,2,4,**1**,4,2,3,2

Three pages of physical memory

OPT



7 total misses, 4 compulsory misses

AMAT = (Tm) + (Miss% * Td)

Assume Tm = 100ns
Assume Td = 1000000 ns (1millisec)

AMAT = ?

100 + 1000000*(7/11) ns